

Control de Velocidad de Motor DC

Hernández Alexis de la Cruz, Jorja Fernando Javier

UTN FRA - Técnicas Digitales III

Avellaneda, Argentina

alexisdelaacruzhernandez@gmail.com

nuevacuentafernando@gmail.com

Abstract— El proyecto realizado consiste en el control de velocidad de un motor DC mediante la aplicación del control PID, el mismo cuenta con ajustes de velocidad final, sentido de giro, tiempo de aceleración y tiempo de desaceleración. Implementa, además, una función de datalogger con un reloj en tiempo real (RTC) y una calibración previa de los parámetros del control PID. También se cuenta con una aplicación web implementada en un sistema basado en Linux para remotizar la adquisición y modificación de parámetros referidos al control. Ha sido desarrollado con un microcontrolador RP2350 (arquitectura Cortex-M33), stick Raspberry Pi Pico 2W, utilizando un sistema operativo en tiempo real (FreeRTOS) y un procesador BCM2712 (arquitectura Cortex-A76) utilizando la placa Raspberry Pi 5 con el sistema operativo de uso general Raspberry Pi OS. Se programó en lenguaje C tanto el módulo de kernel v6.12.47 como el microcontrolador con el entorno de desarrollo Visual Studio Code v1.102.3 con la extensión Raspberry Pi Pico v0.18.0. Para la App web se utilizó Python con FastAPI, html, css y javascript.

I. ÍNDICE

I. ÍNDICE.....	1
II. FUNDAMENTACIÓN.....	1
A. Problemática.....	1
B. Fundamentación Teórica de Hardware.....	1
C. Fundamentación Teórica de Software.....	1
III. OBJETIVOS.....	1
IV. ALCANCE.....	1
IV. DIAGRAMA EN BLOQUES.....	2
V. DIAGRAMA DE SOFTWARE.....	2
VI. DIAGRAMA DE TAREAS.....	2
VII. COMANDOS IMPLEMENTADOS.....	2
A. Comandos Get.....	2
B. Comandos Set.....	3
VIII. CONCLUSIONES.....	3
IX. REFERENCIAS.....	3
VIII. ANEXOS.....	4

II. FUNDAMENTACIÓN

A. Problemática

En muchas aplicaciones se hace necesario implementar un sistema de control de velocidad para motores, con el objetivo de mantener una velocidad constante frente a perturbaciones externas y/o seguir curvas específicas de aceleración y desaceleración. Esto permite mejorar la precisión del sistema y optimizar el consumo energético.

A su vez, hoy en día, es deseable lograr remotizar los sistemas, de forma de poder controlarlos o modificarlos sin tener que interactuar de forma directa con el mismo, contando a su vez con mayor nivel de procesamiento para realizar interfaces gráficas agradables al usuario, por estas razones se implementa la Raspberry Pi 5 al proyecto.

B. Fundamentación Teórica de Hardware

Contando ya con el hardware realizado para la EGA lo que se realizó es una comunicación a través del protocolo UART, de forma de lograr recibir comandos y enviar respuestas hacia la Raspberry Pi 5 [1], la cual cuenta con múltiples puertos UART.

C. Fundamentación Teórica de Software

La ampliación a la EGB se desarrolló sobre un sistema operativo de uso general, específicamente Raspberry Pi OS [2], la cual cuenta con un kernel de Linux en la versión 6.12.47 [3], esto implica desarrollar un driver para el puerto UART y su posterior programación para lograr comunicar el kernel-space con el user-space.

El kernel-space implementa un device tree junto a un char device para manejar el puerto, mientras que en el user-space se escribe y se lee sobre el char device para así poder interactuar por UART.

III. OBJETIVOS

Los objetivos propuestos son los siguientes:

- Desarrollar un driver para el UART.
- Uso de device tree.
- Comunicar un sistema operativo en tiempo real con un sistema operativo de uso general.
- Coordinar programas en kernel-space y user-space.

IV. ALCANCE

- Obtener desde el GPOS los valores de k_p , k_d , k_i , velocidad objetivo, velocidad medida, tiempo de aceleración, tiempo de desaceleración, ciclo de actividad del pwm y el estado de conexión de la memoria SD.
- Modificar desde el GPOS los valores de k_p , k_d , k_i , velocidad objetivo, tiempo de aceleración, tiempo de desaceleración, hora del RTC.
- Desde el GPOS iniciar el control del motor.
- Implementar una aplicación de consola que permita interactuar entre ambos sistemas operativos.
- Implementar una aplicación web con gráficos en tiempo real para interactuar entre ambos sistemas operativos.

IV. DIAGRAMA EN BLOQUES

Los módulos que componen el sistema se muestran en la figura 1, donde se pueden ver las conexiones entre ellos y a que pines corresponden.

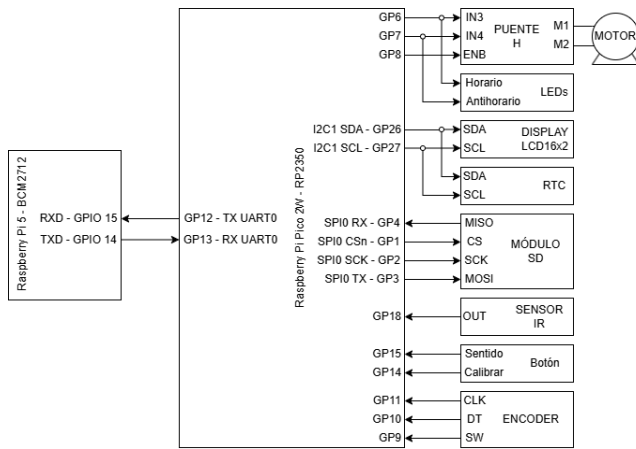


Fig. 1 Diagrama en bloques del control de velocidad

V. DIAGRAMA DE SOFTWARE

En la figura 2 se muestra el diagrama de software implementado.

Podemos observar cómo se relaciona el RTOS (en nuestro caso freeRTOS [4]), corriendo en la Raspberry Pi Pico 2W [5], con el GPOS, corriendo en la Raspberry Pi 5, a través del puerto UART.

En el kernel space del GPOS se implementa un driver de UART, implementado con un device tree y sus respectivas operaciones, junto a esto también se implementa la operación propia del UART para recibir datos. Para lograr la interacción entre el driver y el user space se implementa un char device con sus operaciones de lectura y escritura, donde la escritura envía lo escrito por el puerto UART y la tarea de lectura del archivo devuelve el mensaje recibido por UART.

Para la aplicación de consola se utilizó python y sigue la siguientes pasos:

1. Consulta al usuario si se desea consultar o modificar una variable.
2. Consulta al usuario que variable desea modificar o consultar.
3. En caso de querer modificar también se solicita el nuevo valor a cargar.
4. Con todo consultado se arma el comando y el mismo se escribe sobre el char device.
5. Lee sobre el archivo, recibiendo el valor devuelto por el RTOS.

La aplicación web se implementó con la librería FastAPI [6] de python para manejar el char device, html y css para realizar la implementación gráfica (visualizada en el Anexo 2 - App web) y javascript para indicar a python que realizar con el archivo.

De esta forma tenemos una cadena con las siguientes interacciones:

1. Se detecta la pulsación de un botón y javascript se lo indica al código de python.
2. Python recibe la operación a realizar y configura el comando.
3. Con el comando armado se escribe sobre el char device para enviar por UART.
4. Se lee el char device para obtener la respuesta del RTOS.
5. Se envía la respuesta a javascript y se actualiza la interfaz gráfica.

En el caso del gráfico en tiempo real se implementó un websocket de forma que cada 200ms se escriba sobre el char device los comandos para obtener la velocidad medida y el ciclo de actividad del pwm y, una vez obtenidos, se envían para que el código de javascript actualice el gráfico con los nuevos datos.

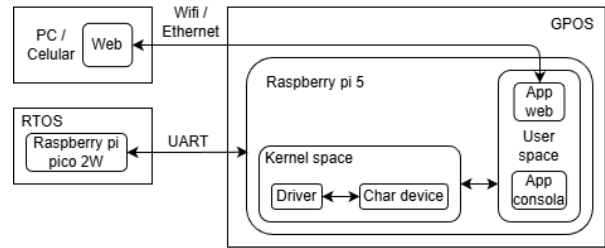


Fig 2. Diagrama de software

VI. DIAGRAMA DE TAREAS

El software implementado en el RTOS consta de 12 tareas, donde 3 de ellas se eliminan al terminar la calibración, el diagrama de tareas que esquematiza la interacción entre cada una de ellas se encuentra en el Anexo 1 - Diagrama de tareas.

Contamos con una tarea central que recibe la velocidad medida, el giro del encoder, la pulsación del encoder y del botón “Sentido”, en base al evento sucedido actualiza una cola con la configuración realizada.

La cola de configuración es leída a su vez por la tarea de control del motor, la del display LCD y la del datalogger, donde estas se desbloquean cada determinado tiempo fijado.

Inicialmente se crea una tarea para configurar la hora del RTC, donde también es necesaria la tarea del encoder y del pulsador calibrar, una vez configurado se crea la tarea de tuning del PID, junto a la que lee la velocidad, la tarea de control y la tarea central.

Una vez finalizada la calibración se crean el resto de tareas y comienza el flujo del programa principal.

La tarea que maneja el puerto UART tiene la propiedad de sobrescribir configuraciones realizadas previamente tanto por los botones, el encoder o la tarea de calibración. De esta forma lo que realiza es la detección del comando y posterior modificación o lectura de variables.

VII. COMANDOS IMPLEMENTADOS

A. Comandos Get

Este tipo de comandos nos permite obtener las configuraciones actuales del sistema, cuenta con los siguientes comandos:

- get pkp: obtiene el valor de kp.
- get pkd: obtiene el valor de kd.
- get pki: obtiene el valor de ki.
- get vob: obtiene la velocidad objetivo.
- get vme: obtiene la velocidad medida.
- get tac: obtiene el tiempo de aceleración.
- get tde: obtiene el tiempo de desaceleración.
- get pwm: obtiene el ciclo de actividad del pwm.
- get mem: obtiene si la SD está conectada o no.

B. Comandos Set

Este tipo de comandos nos permite modificar las configuraciones del sistema e inicializar el control del motor, cuenta con los siguientes comandos:

- set pkp x.xx: escribe sobre kp “x.xx”.
- set pkd x.xxxx: escribe sobre kd “x.xxxx”.
- set pki x.xx: escribe sobre ki “x.xx”.
- set vob ±xxxx: escribe sobre velocidad objetivo “±xxxx”.
- set tac xx.x: escribe sobre tiempo de aceleración “xx.x”.
- set tde xx.x: escribe sobre tiempo de desaceleración “xx.x”.

- set tim HH:MM:SS-dd/mm/ss: configura hora del RTC.
- set son: inicia el control sobre el motor.

VIII. CONCLUSIONES

Si bien en el alcance de velocidad no indicamos velocidades mínimas y máximas nos encontramos con limitaciones físicas de hardware que limitan estas:

- Velocidad mínima: al ensayar al motor obtuvimos una velocidad mínima de funcionamiento de unas 200RPM, siendo esta muy inestable, por lo que se eligió limitar el rango de funcionamiento a un mínimo de 500RPM.
- Velocidad máxima: al medir el motor en vacío y sin puente H obtuvimos las 3000RPM mencionadas anteriormente, al colocar el puente H y realizar nuevamente las mediciones observamos un máximo de aproximadamente 2700RPM, por lo que fijamos el máximo configurable a esta velocidad.
- Frecuencia de PWM: si bien no es mencionado anteriormente, al realizar ensayos sobre el motor encontramos que para una frecuencia de 500Hz el motor se comienza a mover con las 350 cuentas mencionadas. Si se busca aumentar la frecuencia lo que se obtiene es un mayor porcentaje de ciclo de actividad necesario para comenzar el movimiento, junto a una velocidad mínima mayor.

Con respecto al seguimiento de las rampas de aceleración y desaceleración podemos decir que las sigue adecuadamente excepto cuando la velocidad objetivo es cercana al límite máximo, esto se solucionaría aumentando el valor de k_p , pero esto mismo produce mayor error en el seguimiento de las rampas, por lo que se decidió seguir con mayor precisión las rampas pero tener el inconveniente mencionado para altas velocidades.

Nos encontramos con un problema a la hora de implementar el control PID, y es que el tiempo de muestreo para obtener una precisión aceptable en la medición de velocidad es muy largo para implementar ajustes rápidos sobre el control PID, para solucionar esto optamos por realizar 3 cálculos de PID por cada velocidad medida, simplemente cambiando el valor del setpoint, permitiéndonos conseguir un mejor control sobre el motor.

Respecto al sistema operativo de uso general podemos concluir que el mismo realiza de forma correcta los objetivos planteados, se logró remotizar y monitorizar las variables del sistema e implementar gráficos en tiempo real de las variables importantes del sistema, como lo son la velocidad actual y el ciclo de actividad del PWM, gracias a esto podemos observar la respuesta del control PID frente a perturbaciones externas.

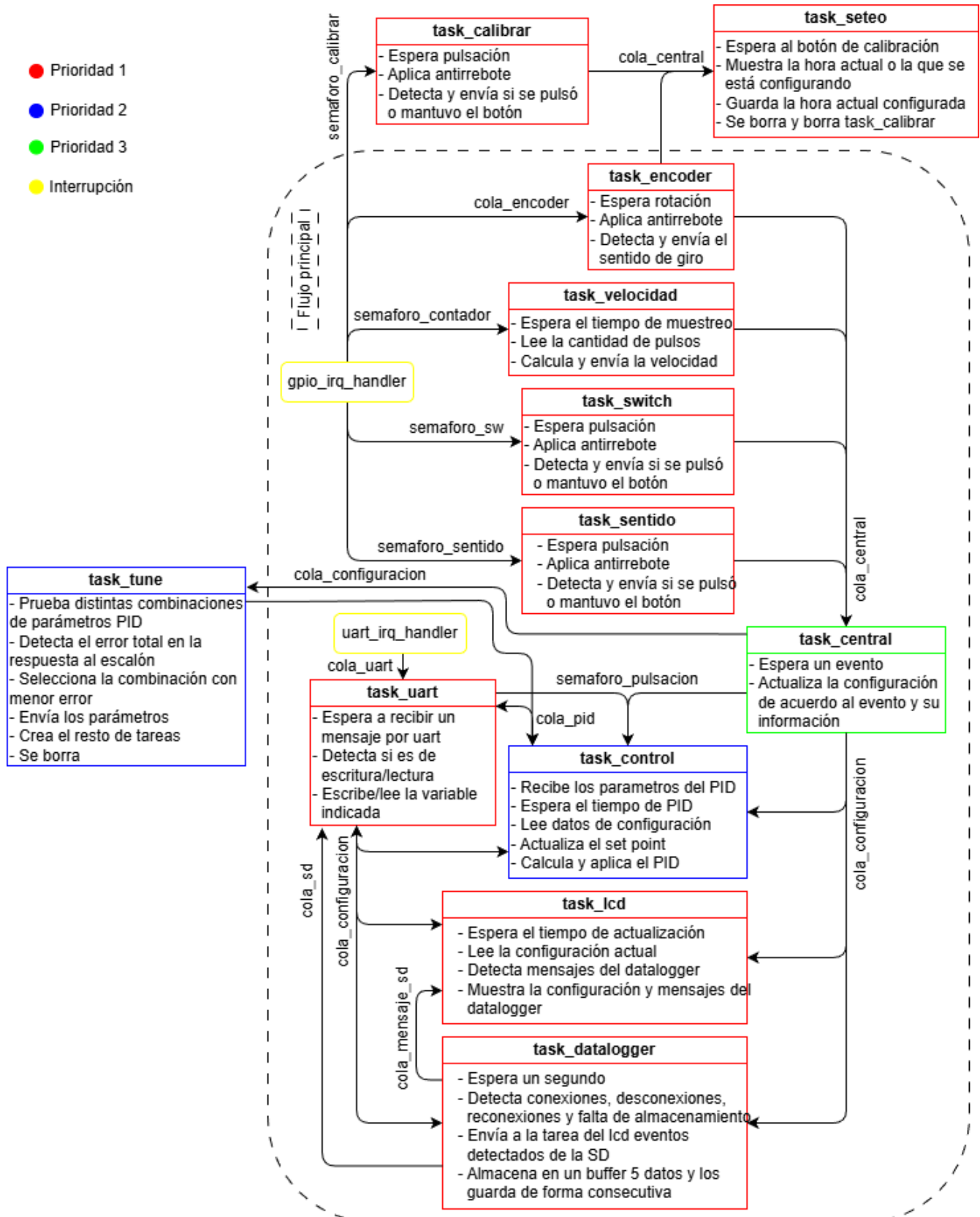
A nivel personal logramos observar la flexibilidad de este tipo de implementación, controlando un sistema con un sistema operativo en tiempo real, que asegure el correcto funcionamiento del proyecto realizado, y visualizando sus datos utilizando un sistema operativo de uso general, permitiendo realizar aplicaciones, tanto a nivel consola como de forma gráfica, interactiva y agradable para el usuario.

IX. REFERENCIAS

- [1] Raspberry Pi 5
<https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>
- [2] Raspberry Pi OS
<https://www.raspberrypi.com/software/operating-systems/>
- [3] Kernel Linux v6.12.47
<https://elixir.bootlin.com/linux/v6.12.47/source>
- [4] FreeRTOS
<https://www.freertos.org>
- [5] Raspberry Pi Pico 2W
<https://pip-assets.raspberrypi.com/categories/1088-raspberry-pi-pico-2-w/documents/RP-008304-DS-1-pico-2-w-datasheet.pdf?disposition=inline>
- [6] FastAPI
<https://fastapi.tiangolo.com/>

VIII. ANEXOS

ANEXO 1 - Diagrama de Tareas



Fecha y hora

Fecha:
dd/mm/aaaa

Hora:
--:--:--

Enviar

Control de Motor DC

Parametros PID

Kp

0.50

Ki

0.75

Kd

0.0050

Configuración

Velocidad objetivo [RPM]

1500

Tiempo de aceleración [seg]

5.0

Tiempo de desaceleración [seg]

5.0

Iniciar control

Consultar SD

Estado SD: off

Iniciar gráfico

